DEVEOPS LAB EXAM

1. Explain the process of creating a fresh Git repository, storing files locally, and pushing them to GitHub, along with the commands.

Theory:

Git is a distributed version control system used to track changes in files and manage source code. GitHub is a cloud-based platform that hosts Git repositories and enables developers to collaborate on projects.

Steps involved

1. Create a local repository: Create a project folder on the local machine to store files.
2. Initialize Git: Use `git init` to initialize a new Git repository in the folder.
3. Create files: Add the required files to the project directory.
4. Stage files: Use `git add` to select the files that need to be committed.
5. Commit changes: Use `git commit` to save a snapshot of the staged files in the local repository.
6. Create a GitHub repository: Create a new repository on GitHub to store the project online.
7. Connect the remote repository: Use `git remote add origin` to link the local repository with the GitHub repository.
8. Push files to GitHub: Use `git push` to upload the committed changes to GitHub.

Important Git commands

Command	Purpose
`git init`	Creates a new local Git repository.
`git add .`	Stages all changes in the current directory.
`git commit -m "message"`	Saves staged changes in the repository.

`git remote add origin URL`	Connects the local repository to GitHub.
`git push -u origin main`	Pushes the `main` branch to GitHub.

Advantages of Git and GitHub

- Version control: Tracks changes made to files over time.
- Backup: Stores project code remotely on GitHub.
- Collaboration: Allows multiple developers to work on the same project.
- Branching and merging: Supports parallel development and combining changes.
- Change recovery: Allows developers to inspect previous commits and restore earlier versions.

Practical Part:

1. Aim

To create a fresh Git repository on a local machine, add and commit a text file, connect the repository to GitHub, and push the file to the remote repository.

3. Project structure

GitPractice/

├── .git/

└── file1.txt

- `.git/` stores repository metadata and version history.
- `file1.txt` contains the project text.

4. Procedure and commands

git config --global user.name "Pranati Rao Pinisetti"
git config --global user.email "YOUR_GITHUB_EMAIL"

Step 1: Create the project directory.

mkdir GitPractice

cd GitPractice

Step 2: Initialize the Git repository.

git init

Step 3: Create a text file.

echo "Hello GitHub" > file1.txt

Step 4: Stage the file.

git add file1.txt

Step 5: Commit the file.

git commit -m "Initial commit"

Step 6: Rename the branch to `main`.

git branch -M main

Step 7: Create an empty repository named `GitPractice` on GitHub.

Open GitHub — New Repository, enter the repository name, and click Create repository. Do not initialize it with a README.

Step 8: Connect the local repository to GitHub.

git remote add origin https://github.com/YOUR_USERNAME/GitPractice.git

Replace `YOUR_USERNAME` with your GitHub username.

Step 9: Push the file to GitHub.

git push -u origin main

5. Code

The contents of `file1.txt` are:

Hello GitHub

6. Output

After successful execution:

- The local Git repository is created.
- The file is committed locally.
- The local repository is connected to GitHub.
- `file1.txt` is visible in the GitHub repository.
- `git status` displays `nothing to commit, working tree clean`.

7. Result

A fresh Git repository was created successfully, and the text file was stored locally and pushed to GitHub.

2. Create a text file on the local machine, save it in a local repository, and push it to GitHub.

Theory:

Git is a distributed version control system used to track changes in files, while GitHub is an online platform used to host Git repositories.

Creating a text file locally and pushing it to GitHub involves the following steps:

1. Create a local repository: Create a project directory and initialize it using `git init`.
2. Create a text file: Create a file containing the required data.
3. Check repository status: Use `git status` to identify untracked files.
4. Stage the file: Use `git add filename` to prepare the file for committing.
5. Commit the file: Use `git commit -m "message"` to save the changes in the local repository.
6. Create a remote repository: Create an empty repository on GitHub.
7. Connect the repositories: Use `git remote add origin URL` to connect the local repository to GitHub.
8. Push the file: Use `git push -u origin main` to upload the committed file to GitHub.

Important commands

Command	Purpose
`git init`	Initializes a local repository
`git add student.txt`	Stages the text file
`git commit -m "message"`	Commits the file

`git remote add origin URL`	Connects the GitHub repository
`git push -u origin main`	Uploads commits to GitHub

Advantages:

- Maintains a history of changes.
- Provides remote storage for project files.
- Supports collaboration and version control.
- Allows previous versions of files to be inspected.

Conclusion: A text file can be stored in a local Git repository and uploaded to GitHub by staging, committing, connecting the remote repository, and pushing the changes.

Part B: Practical Implementation

1. Aim

To create a text file on the local machine, save it in a local Git repository, and push it to a remote GitHub repository.

2. Requirements

- Git installed on the system.
- GitHub account.
- Git Bash or VS Code terminal.
- Internet connection.

3. Project Structure

GitQuestion2/

├── .git/

└── student.txt

4. Procedure and commands

Step 1: Create the folder and initialize Git.

mkdir GitQuestion2

cd GitQuestion2

git init

Step 2: Create the text file.

echo "Welcome to Git and GitHub" > student.txt

Step 3: Stage the file.

git add student.txt

Step 4: Commit the file.

git commit -m "Add student text file"

Step 5: Create an empty GitHub repository.

Open GitHub — New Repository and create a repository named `GitQuestion2`. Leave the README option unchecked.

Step 6: Connect the local repository to GitHub.

git branch -M main

git remote add origin https://github.com/YOUR_USERNAME/GitQuestion2.git

Replace `YOUR_USERNAME` with your GitHub username.

Step 7: Push the file to GitHub.

git push -u origin main

5. Code

Contents of `student.txt`:

Welcome to Git and GitHub

6. Output

The `student.txt` file is created, committed to the local repository, and uploaded to GitHub. It becomes visible in the `GitQuestion2` repository.

Verify using:

git status

git log --oneline

Expected status:

nothing to commit, working tree clean

7. Result

The text file was successfully created, stored in the local Git repository, and pushed to GitHub.

3. Create another text file in GitHub and pull it into the local repository.

Theory:

Git is a version control system used to track changes in files, while GitHub is an online platform used to store and manage Git repositories.

Sometimes, files are created directly on GitHub and need to be downloaded to the local computer. The `git pull` command is used to fetch and integrate changes from the remote repository into the local branch.

The procedure involves creating a file on GitHub, adding its content, and committing the changes. Then, the `git pull origin main` command is executed in the local repository to retrieve the file.

Advantages:

- Keeps local files updated with GitHub.
- Makes remote changes available locally.
- Supports collaboration between developers.

Conclusion: The `git pull` command helps synchronize the local repository with changes made on GitHub.

Part B: Practical Implementation

1. Aim

To create a text file on GitHub and pull it into the local Git repository.

3. Project Structure

Before pulling:

GitQuestion3/

└── .git/

After pulling:

GitQuestion3/

├── .git/

└── github.txt

4. Procedure and Commands

Step 1: Open your existing local repository.

cd GitQuestion3

If you completed Question 2 using `GitQuestion2`, you can use that repository instead.

Step 2: Create a file on GitHub.

1. **Open your GitHub repository.**
2. **Click Add file → Create new file.**
3. **Enter the filename `github.txt`.**
4. **Add the following content:**

This file was created on GitHub.

5. Click Commit changes and commit it to the `main` branch.

Step 3: Pull the changes into the local repository.

git pull origin main

This downloads and integrates the latest changes from the remote `main` branch.

Step 4: Verify the file.

This file was created on GitHub.

5. Code

Contents of `github.txt`:

This file was created on GitHub.

6. Output

The `github.txt` file created on GitHub is downloaded and appears in the local project directory.

7. Result

The file was successfully created on GitHub and retrieved into the local repository using `git pull origin main`.

4. Create a text file in the `main` branch and save it in the local repository.

Part A: Theory

Git is a version control system that allows developers to track changes and manage different versions of a project. A branch is an independent line of development in Git. The `main` branch is commonly used as the primary branch of a repository.

To create a text file in the `main` branch, we first initialize a Git repository and switch to the `main` branch. Then, we create a text file, stage it using `git add`, and save it in the repository using `git commit`.

This process helps maintain files and their version history in the main branch.

Advantages:

- Tracks file changes.
- Maintains project history.
- Keeps the main branch organized.

Conclusion: A text file can be created and saved in the `main` branch using Git commands such as `git add` and `git commit`.

Part B: Practical Implementation

1. Aim

To create a text file in the `main` branch and save it in the local Git repository.

3. Project Structure

GitQuestion4/

├── .git/

└── mainfile.txt

4. Procedure and Commands

Step 1: Create a folder and initialize Git.

mkdir GitQuestion4

cd GitQuestion4

git init

Step 2: Create and switch to the `main` branch.

git branch -M main

Step 3: Create a text file.

echo "This file belongs to the main branch" > mainfile.txt

Step 4: Stage the file.

git add mainfile.txt

Step 5: Commit the file.

git commit -m "Add file to main branch"

Step 6: Verify the branch and file.

git branch

git status

ls

5. Code

Contents of `mainfile.txt`:

This file belongs to the main branch

6. Output

- The `mainfile.txt` file is created.
- The file is committed to the `main` branch.
- The working tree is clean after the commit.

7. Result

A text file was successfully created and saved in the local Git repository's `main` branch.

5. Create another text file in the `branch-1` branch and save it in the local repository.

Part A: Theory

Git is a version control system that allows developers to maintain different versions of a project using branches. A branch is a separate line of development that allows changes to be made independently without directly affecting other branches.

To create a file in the `branch-1` branch, we first create a Git repository and switch to the new branch using `git checkout -b branch-1`. Then, we create a text file, stage it using `git add`, and save it using `git commit`.

This allows us to work on separate features independently and merge them into the main branch later.

Advantages:

- Supports parallel development.
- Keeps changes separate from the main branch.
- Makes merging changes easier.

Conclusion: Git branches allow developers to create and save files independently before merging them into the main branch.

Part B: Practical Implementation

1. Aim

To create a text file in the `branch-1` branch and save it in the local Git repository.

3. Project Structure

GitQuestion4/

├── .git/

├── mainfile.txt

└── branchfile.txt

4. Procedure and Commands

Step 1: Open the repository created in Question 4.

cd GitQuestion4

Step 2: Create and switch to `branch-1`.

git checkout -b branch-1

This creates the branch and switches to it.

Step 3: Create a text file.

echo "This file belongs to branch-1" > branchfile.txt

Step 4: Stage the file.

git add branchfile.txt

Step 5: Commit the file.

git commit -m "Add file to branch-1"

Step 6: Verify the branch and files.

git branch

git status

ls

git checkout main - to switch main branch

The active branch will be marked with `*` in the `git branch` output.

5. Code

Contents of `branchfile.txt`:

This file belongs to branch-1

6. Output

- The `branch-1` branch is created.
- `branchfile.txt` is committed to that branch.
- The file `mainfile.txt` from the main branch is also present because the new branch was created from it.

7. Result

A text file was successfully created and committed to the `branch-1` branch of the local Git repository.

6. Merge the two files into the `main` branch and perform the same operation in GitHub.

Part A: Theory

Git merging is the process of combining changes from one branch into another. It allows developers to integrate files developed separately into a single branch.

To merge `branch-1` into `main`, we first switch to the `main` branch using `git checkout main`. Then, we execute `git merge branch-1` to combine the changes. If there are no conflicts, Git completes the merge automatically.

To perform the same operation on GitHub, push both branches to the remote repository and create a pull request to merge `branch-1` into `main`.

Advantages:

- Combines work from different branches.
- Supports team collaboration.
- Maintains project history.

Conclusion: Git merge combines changes from different branches into the main branch.

Part B: Practical Implementation

1. Aim

To merge the files from `branch-1` into `main` locally and perform the same operation on GitHub.

3. Project Structure

After merging:

GitQuestion4/

├── .git/

├── mainfile.txt

└── branchfile.txt

4. Procedure and Commands

Step 1: Switch to the main branch.

git checkout main

Step 2: Merge `branch-1` into `main`.

git merge branch-1

Step 3: Verify the files.

ls

git log --oneline

Both `mainfile.txt` and `branchfile.txt` should now be present in the `main` branch.

Step 4: Push both branches to GitHub.

If the remote repository is already configured, run:

git push -u origin main

git push -u origin branch-1

If you have not connected a remote repository yet, add it first:

git remote add origin https://github.com/YOUR_USERNAME/GitQuestion4.git

Replace `YOUR_USERNAME` with your GitHub username.

Step 5: Merge on GitHub.

1. Open your repository on GitHub.
2. Select Pull requests → New pull request.
3. Set the base branch to `main` and compare branch to `branch-1`.
4. Click Create pull request.
5. Click Merge pull request, then confirm the merge.

If GitHub reports that the branches are already merged, the local merge has already been pushed to `main`. No additional merge is needed.

5. Code

`mainfile.txt`

This file belongs to the main branch

`branchfile.txt`

This file belongs to branch-1

6. Output

Both files are available in the `main` branch after the merge. The GitHub repository also reflects the merged changes.

7. Result

The changes from `branch-1` were successfully merged into `main` in the local repository and synchronized with GitHub.

7. Explain the process of creating a Docker container using Docker commands.

Part A: Theory

Docker is a platform used to package and run applications in containers. A Docker image is a template containing the application and its required environment, while a Docker container is a running instance of that image.

The process begins by installing Docker and ensuring the Docker Engine is running. The required image is downloaded using `docker pull`, and a container is created and started using `docker run`. Commands such as `docker ps` and `docker ps -a` are used to check running and stopped containers.

Advantages:

- Provides an isolated environment for applications.
- Reduces dependency and compatibility problems.
- Makes application deployment easier.

Conclusion: Docker commands help download images, create containers, and manage their execution.

Part B: Practical Implementation

1. Aim

To download a Docker image, create a container, run it, and verify its status using Docker commands.

3. Project Structure

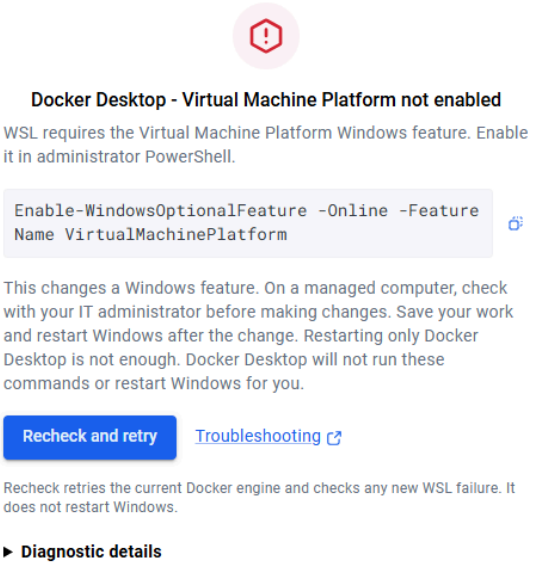
No project folder or application code is required for this example.

Docker

└── Image: hello-world

└── Container: mycontainer

4. Procedure and commands



C1:**Enable-WindowsOptionalFeature -Online -FeatureName VirtualMachinePlatform -All**
Restart and run: wsl - - install (if it works then dont do c2)
C2: **dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-Linux /all /norestart**

dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all /norestart
C3: **wsl - - install**
C4: Get-WindowsOptionalFeature -Online -FeatureName VirtualMachinePlatform
Restart
c5:**wsl --list --verbose**

Step 1: Check the Docker installation.

docker --version

This displays the installed Docker version.

Step 2: Check whether Docker Engine is running.

docker info

If this command succeeds, Docker is ready to run containers.

Step 3: Download the image.

docker pull hello-world

This downloads the `hello-world` image from Docker Hub.

Step 4: Create and run a container.

docker run --name mycontainer hello-world

Explanation:

- `docker run` creates and starts a container.
- `--name mycontainer` assigns a name to the container.
- `hello-world` specifies the image.

The container displays a message and then stops automatically. This is expected behavior for this image.

Step 5: Check all containers.

docker ps -a

This displays running and stopped containers. You should see `mycontainer` with an exited status.

Step 6: Check downloaded images.

docker images

This lists the images stored on your computer.

5. Code

No application code is required. The following Docker command performs the main operation:

docker run --name mycontainer hello-world

6. Output

The terminal displays a message similar to:

Hello from Docker!

This message shows that your installation

appears to be working correctly.

The container appears in `docker ps -a` with an exited status.

7. Result

The Docker image was downloaded successfully, and a container named `mycontainer` was created and executed using Docker commands.

8. Write Docker commands to pull the latest Docker image for a specified image.

Part A: Theory

Docker is a platform used to create and run applications inside containers. A Docker image is a template used to create a container. Images can be downloaded from Docker Hub, an online repository for Docker images.

The `docker pull` command is used to download an image from Docker Hub to the local machine. To download the latest version of an image, we can use the `latest` tag. For example, `docker pull nginx:latest` downloads the latest tagged Nginx image.

After downloading the image, we can verify it using `docker images` and create a container using `docker run`.

Advantages:

- Downloads images when needed.
- Makes application deployment easier.
- Allows images to be reused to create multiple containers.

Conclusion: The `docker pull` command downloads Docker images from Docker Hub for local use.

Part B: Practical Implementation

1. Aim

To download the latest Docker image from Docker Hub and verify that it is available locally.

3. Project Structure

No project folder or application code is required.

Docker

└── Local Images

  └── nginx:latest

4. Procedure and Commands

Step 1: Check Docker.

docker --version

Step 2: Pull the latest Nginx image.

docker pull nginx:latest

Here:

- `docker pull` downloads an image.
- `nginx` is the image name.
- `latest` specifies the image tag.

Step 3: Verify the downloaded image.

docker images

Expected output will be similar to:

REPOSITORY TAG IMAGE ID CREATED SIZE

nginx latest abc123456789 ... ...

The image ID, creation date, and size may differ.

Step 4: Run a container using the image (optional).

docker run -d --name mynginx -p 8080:80 nginx:latest

Open `http://localhost:8080` in your browser. You should see the Nginx welcome page.

5. Code

The main command is:

docker pull nginx:latest

6. Output

The terminal displays download progress and confirms that the image has been pulled. The image appears in the output of `docker images`.

7. Result

The latest tagged Nginx Docker image was successfully downloaded from Docker Hub and verified on the local machine.

Remember for the exam:

- `docker pull nginx:latest` — downloads the image.
- `docker images` — lists downloaded images.
- `docker run` — creates and starts a container from an image.

9. Perform the following Docker operations:

- **Create a Docker container named `MyPostgresContainer`, set the `POSTGRES_PASSWORD` environment variable to `abcd1234`, run the container on port `8844`, and pull the latest `traefik/whoami` image into the local machine.**
- **Run two containers on two different ports: `8686` and `8787`.**
- **Execute the following commands and explain the differences:**
  - **`curl http://localhost:8686`**
  - **`curl http://localhost:8687`**

Part A: Theory

Docker is a platform used to run applications inside containers. A container is created from a Docker image and can be configured using environment variables and port mapping.

- Environment variable (`-e`): Passes configuration values to a container, such as a PostgreSQL password.
- Port mapping (`-p`): Connects a port on the computer to a port inside the container. For example, `-p 8844:5432` maps host port `8844` to PostgreSQL's container port `5432`.
- Docker image: A template used to create containers. The `docker pull` command downloads an image from Docker Hub.
- `curl`: Sends an HTTP request to a server and displays its response.

Two containers can run on different host ports, allowing us to access their services separately. For example, ports `8686` and `8787` can map to port `80` inside two different web-server containers.

Conclusion: Docker commands allow us to configure containers, map ports, download images, and test services using `curl`.

Part B: Practical Implementation

1. Aim

To create a PostgreSQL container with a password and port mapping, pull the `traefik/whoami` image, run two web containers on different ports, and compare their responses using `curl`.

3. Project structure

No source-code files are required for this exercise. All operations are performed using Docker commands.

Docker

├── MyPostgresContainer → Host port 8844

├── Whoami1 → Host port 8686

└── Whoami2 → Host port 8787

4. Procedure and commands

Step 1: Create the PostgreSQL container

Run:

docker run -d --name MyPostgresContainer -e POSTGRES_PASSWORD=abcd1234 -p 8844:5432 postgres:latest

Explanation:

Command	Purpose
`docker run`	Creates and starts a container
`-d`	Runs it in the background
`--name MyPostgresContainer`	Sets the container name
`-e POSTGRES_PASSWORD=abcd1234`	Sets the PostgreSQL password
`-p 8844:5432`	Maps host port 8844 to container port 5432

`postgres:latest`	Specifies the PostgreSQL image

Verify the container:

docker ps

Step 2: Pull the latest Traefik Whoami image

docker pull traefik/whoami:latest

Verify the image:

docker images

The `traefik/whoami` image runs a small HTTP server that returns information about the container handling the request.

Step 3: Run the first container on port 8686

docker run -d --name Whoami1 -p 8686:80 traefik/whoami:latest

This maps host port `8686` to container port `80`.

Step 4: Run the second container on port 8787

docker run -d --name Whoami2 -p 8787:80 traefik/whoami:latest

This maps host port `8787` to container port `80`.

Both containers use the same image but have different names and host ports.

Step 5: Check all running containers

docker ps

You should see three containers:

- `MyPostgresContainer`
- `Whoami1`
- `Whoami2`

Check that PostgreSQL has finished starting before testing database connections. The `whoami` containers should be ready to answer HTTP requests shortly after starting.

Step 6: Execute the first curl command

curl.exe http://localhost:8686

Expected response will be similar to:

Hostname: <container-specific-id>

IP: <container-ip>

...

The hostname and other details identify the container that handled the request.

Step 7: Execute the second curl command

curl.exe http://localhost:8787

This returns information about the second container, including its own hostname.

Why use `curl.exe`? In Windows PowerShell, `curl` may be an alias for another command. Using `curl.exe` ensures that the actual curl program is executed.

5. Code / Commands summary

```
docker run -d --name MyPostgresContainer -e POSTGRES_PASSWORD=abcd1234 -p 8844:5432 postgres:latest


docker pull traefik/whoami:latest


docker run -d --name Whoami1 -p 8686:80 traefik/whoami:latest


docker run -d --name Whoami2 -p 8787:80 traefik/whoami:latest


docker ps


curl.exe http://localhost:8686
```

curl.exe http://localhost:8787

6. Output and difference between the curl commands

Command	What happens
`curl.exe http://localhost:8686`	Sends an HTTP request to `Whoami1`.
`curl.exe http://localhost:8787`	Sends an HTTP request to `Whoami2`.

Both requests should return HTTP responses from the Whoami application. The hostname and container-related details will generally differ because each request is handled by a different container.

7. Result

A PostgreSQL container was configured with the specified password and port mapping. The `traefik/whoami` image was downloaded, two web containers were started on ports `8686` and `8787`, and their responses were tested using `curl`.

10. Explain the following HTTP verbs: GET, POST, DELETE, and PUT.

Part A: Theory

HTTP verbs (methods) specify the type of operation a client wants to perform on a server resource. They are commonly used in REST APIs to communicate between applications and servers.

The four important HTTP methods are:

1. GET: Used to retrieve or read data from the server without modifying it.
2. POST: Used to send data to the server, commonly to create a new resource.
3. PUT: Used to create or completely update a resource at a specified URL. It generally replaces the existing resource representation.
4. DELETE: Used to remove a specified resource from the server.

These methods help APIs perform different operations in a standard way.

HTTP method	Purpose	Example
GET	Retrieve data	View student details
POST	Create a resource	Add a new student
PUT	Replace or update a resource	Update student details
DELETE	Remove a resource	Delete a student

Conclusion: GET, POST, PUT, and DELETE are HTTP methods used to retrieve, create, update, and remove resources through APIs.

11. Write a microservice to implement the above HTTP operations for the following data structure, orchestrate it using Kubernetes (K8s), and test the Pod using Postman:

- **`id`: Integer**
- **`studentName`: String**
- **`totalMarks`: Integer**

Step 2: Check Docker Desktop

1. Open Docker Desktop.
2. Go to Settings → Kubernetes.
3. If you see an option to enable Kubernetes, enable it and click Apply & Restart.
4. Wait until Kubernetes shows that it is running.

If you do not see a Kubernetes option, tell me what appears in your Docker Desktop Settings.

Step 3: Select the Kubernetes context

After Kubernetes starts, run:

kubectl config get-contexts

If `docker-desktop` appears, select it:

kubectl config use-context docker-desktop

Then verify:

kubectl cluster-info

kubectl get nodes

You should see a Kubernetes node in the `Ready` state.

A. Theory

A microservice is a small, independent application that performs a specific task. Here, the student microservice manages student records using the HTTP methods GET, POST, PUT, and DELETE.

Each student record contains:

- `id` – Integer
- `studentName` – String

- `totalMarks` – Integer

Kubernetes (K8s) is a container orchestration platform used to deploy, manage, and expose containerized applications. A Kubernetes Deployment manages application Pods, while a Service provides access to those Pods.

The microservice is tested using Postman, which sends HTTP requests and displays server responses.

HTTP method	Operation	Example
GET	Retrieve student records	View all students
POST	Create a student record	Add a student
PUT	Update a student record	Change marks
DELETE	Delete a student record	Remove a student

B. Practical

Aim

To develop a student microservice supporting GET, POST, PUT, and DELETE, deploy it using Kubernetes, and test the API using Postman.

Requirements

- Python 3
- Flask
- Docker Desktop with Kubernetes enabled
- Postman
- VS Code

Project structure

StudentMicroservice/

├── app.py

├── requirements.txt

├── Dockerfile

└── k8s.yml

1. Create `app.py`

```
from flask import Flask, request, jsonify


app = Flask(__name__)


students = []


# GET: Retrieve all students
@app.route('/students', methods=['GET'])
def get_students():
    return jsonify(students), 200



# POST: Add a new student
@app.route('/students', methods=['POST'])
def add_student():
    data = request.get_json()


    if not data or not all(
```

```
        k in data for k in ('id', 'studentName', 'totalMarks')
    ):
        return jsonify({"error": "Missing student fields"}), 400

    try:
        student_id = int(data["id"])
        total_marks = int(data["totalMarks"])
    except (ValueError, TypeError):
        return jsonify({"error": "id and totalMarks must be integers"}), 400

    if any(s["id"] == student_id for s in students):
        return jsonify({"error": "Student ID already exists"}), 409

    student = {
        "id": student_id,
        "studentName": str(data["studentName"]),
        "totalMarks": total_marks
    }

    students.append(student)
    return jsonify(student), 201


# PUT: Update an existing student
@app.route('/students/<int:id>', methods=['PUT'])
```

```
def update_student(id):
    data = request.get_json() or {}

    for student in students:
        if student["id"] == id:
            try:
                if "totalMarks" in data:
                    student["totalMarks"] = int(data["totalMarks"])
            except (ValueError, TypeError):
                return jsonify({"error": "totalMarks must be an integer"}), 400

            if "studentName" in data:
                student["studentName"] = str(data["studentName"])

            return jsonify(student), 200

    return jsonify({"error": "Student not found"}), 404



# DELETE: Remove a student
@app.route('/students/<int:id>', methods=['DELETE'])
def delete_student(id):
    for student in students:
        if student["id"] == id:
            students.remove(student)
```

```
            return jsonify({"message": "Student deleted successfully"}), 200

    return jsonify({"error": "Student not found"}), 404



if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

2. Create `requirements.txt`

```
Flask
```

3. Create `Dockerfile`

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
```

4. Create `k8s.yml`

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: student-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: student
  template:
    metadata:
      labels:
        app: student
    spec:
      containers:
        - name: student-container
          image: student-microservice:1.0
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 5000
---
apiVersion: v1
kind: Service
metadata:
```

```
  name: student-service
spec:
  selector:
    app: student
  type: NodePort
  ports:
    - port: 5000
      targetPort: 5000
      nodePort: 30080
```

5. Build and deploy the application

Open the VS Code terminal inside the `StudentMicroservice` folder.

Step 1: Build the Docker image.

docker build -t student-microservice:1.0 .

Step 2: Ensure Docker Desktop Kubernetes is enabled and running. Then apply the Kubernetes configuration:

kubectl apply -f k8s.yml

Step 3: Verify the deployment and Pod.

kubectl get deployments

kubectl get pods

kubectl get services

Wait until the Pod shows `Running`.

Step 4: Forward the Service port.

kubectl port-forward service/student-service 5000:5000

Keep this terminal running. The API is now accessible at `http://localhost:5000`.

Install postman : https://www.postman.com/downloads/

1. Open Postman.
2. Click New and select HTTP (or open a new HTTP request tab).
3. You will see a method dropdown, a URL field, and a Send button.
4. Select the HTTP method and enter the URL shown below.

Let's test the four operations individually.

Step 4: Test GET — View all students

GET

Retrieve students

Enter this URL in Postman:

`http://localhost:5000/students`

Copy URL

1. Select GET from the dropdown.
2. Enter the URL.
3. Click Send.
   Expected response the first time:

[]

200 OK

Step 5: Test POST — Add a student

POST

Create a student

URL: `http://localhost:5000/students`

1. Change the method to POST.
2. Click the Body tab.
3. Select raw.
4. Select JSON from the format dropdown.
5. Paste the following data.

```
{
  "id": 1,
```

"studentName": "Rahul",

"totalMarks": 450

}

6. Click Send.

Expected response:

{

"id": 1,

"studentName": "Rahul",

"totalMarks": 450

}

201 Created

Step 6: Test PUT — Update student marks

PUT

Update a student

URL: `http://localhost:5000/students/1`

1. Select PUT.
2. Open Body → raw → JSON.
3. Enter:

{

"studentName": "Rahul",

"totalMarks": 480

}

4. Click Send.

Expected response: the student's marks change from `450` to `480`.

200 OK

Step 7: Test DELETE — Remove a student

DELETE

Delete a student

URL: `http://localhost:5000/students/1`

1. Select DELETE.
2. You do not need to enter a request body.
3. Click Send.

Expected response:

{

"message": "Student deleted successfully"

}

200 OK

12. Explain the process of creating a Docker container using a Dockerfile.

A. Theory

A Dockerfile is a text file containing instructions to build a Docker image. A Docker image acts as a template, and a container is an instance created from that image.

The process involves creating a Dockerfile, building an image using `docker build`, and creating and running a container using `docker run`.

B. Practical

Aim

To create a Docker image using a Dockerfile and run a Docker container from that image.

Requirements

- Docker Desktop
- VS Code or any text editor
- Terminal (PowerShell or Command Prompt)

Project structure

DockerProject/

├── Dockerfile

└── index.html

Step 1: Create `index.html`

```
<html>

<body>

    <h1>Hello from Docker Container!</h1>

</body>

</html>
```

Step 2: Create `Dockerfile`

```
FROM nginx:latest
```

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80

Explanation:

- `FROM` specifies the base image.
- `COPY` copies the HTML file into the image.
- `EXPOSE` documents the port used by the application.

Step 3: Build the Docker image

Open the terminal inside the `DockerProject` folder and execute:

docker build -t mydockerimage .

Here, `-t` assigns a name to the image, and `.` specifies the current directory as the build context.

Step 4: Create and run the container

docker run -d --name mydockercontainer -p 8080:80 mydockerimage

Here, `-d` runs the container in the background, and `-p 8080:80` maps host port 8080 to container port 80.

Step 5: Verify the container

docker ps

docker images

Open the browser and visit:

`http://localhost:8080`

Output

The browser displays:

Hello from Docker Container!

The Docker image `mydockerimage` and the running container `mydockercontainer` can also be verified using the commands above.

Result

A Docker image was successfully built using a Dockerfile, and a container was created and run from that image.

13. Perform the following operations:

- **Pull the latest image of `jkaninda/simple-api` into the local machine.**
- **Download the `jkaninda-simple-api-docker-compose` file from the provided link.**
- **Run the command `docker compose -f jkaninda-simple-api-docker-compose up -d`.**
- **Test the service using Postman:**
  - **GET on `/`**
  - **GET on `/books`**

1. Aim

To pull the latest `jkaninda/simple-api` Docker image, run the application using Docker Compose, and test the `GET /` and `GET /books` endpoints using VS Code REST Client.

2. Requirements

- Docker Desktop
- VS Code
- REST Client extension in VS Code
- Internet connection

No Postman installation is required.

3. Project structure

Create a folder named `Question13` in VS Code.

```
Question13/
├── jkaninda-simple-api-docker-compose
```

4. Procedure

Step 1: Open the project in VS Code

1. Create a folder named `Question13`.
2. Open VS Code.
3. Select File → Open Folder and choose `Question13`.

4. Open the terminal using Terminal → New Terminal.

Check that Docker is running:

docker --version

docker info

If `docker info` displays server information without an error, continue.

Step 2: Pull the latest Docker image

Run:

docker pull jkaninda/simple-api:latest

Verify the image:

docker images

Expected output: an image named `jkaninda/simple-api` with the tag `latest`.

Step 3: Create the Docker Compose file

Create a file named exactly:

jkaninda-simple-api-docker-compose

Do not add a `.txt` extension.

Paste the following YAML configuration:

```
services:
  simple-api:
    container_name: simple-api
    image: jkaninda/simple-api:latest
    restart: unless-stopped
```

```
    ports:
      - "8080:8080"
    environment:
      SPRING_DATA_REDIS_HOST: redis
      SPRING_DATA_REDIS_PORT: 6379
      SPRING_DATA_REDIS_PASSWORD: password
      RUN_DATA_SEED: "true"
    depends_on:
      - redis

  redis:
    image: redis:alpine
    container_name: redis
    restart: unless-stopped
    command: redis-server --requirepass password
```

Save the file with Ctrl + S.

Purpose of the configuration:

- image: Specifies the Docker image used by the API.
- ports: Maps host port 8080 to container port 8080.
- SPRING_DATA_REDIS_HOST: Connects the API to the Redis service.
- depends_on: Starts the Redis service before the API container.
- RUN_DATA_SEED: Requests initial sample data seeding.

The Redis service is necessary for the /books endpoint to retrieve book records in this setup.

Step 4: Start the containers

Run the required command from the Question13 folder:

docker compose -f jkaninda-simple-api-docker-compose up -d

The -d option runs the services in the background.

Check the running containers:

docker ps

Expected containers:

Container	Purpose
simple-api	Runs the REST API
redis	Stores the book data

Both should show a running status.

If the containers are not running, inspect the logs:

docker logs simple-api

docker logs redis

Test 1: GET root endpoint

GET http://localhost:8080/

```
### Test 2: GET all books

GET http://localhost:8080/books
```

Save the file.

5. Output

The following results were obtained during testing:

6. Result

The latest jkaninda/simple-api Docker image was pulled, the API and Redis services were started using Docker Compose, and both GET endpoints were successfully tested using VS Code REST Client.

14. Write commands for creating Deployments, Services, and Pods in Kubernetes (K8s).

A. Theory

Kubernetes (K8s) is a container orchestration platform used to deploy, manage, and scale containerized applications.

- Pod: The smallest deployable unit in Kubernetes. It contains one or more containers.
- Deployment: Manages Pods and maintains the desired number of replicas.
- Service: Exposes an application running in Pods and allows access to it within or outside the cluster.

B. Practical

Aim

To create and manage Deployments, Services, and Pods using Kubernetes commands.

Requirements

- Docker Desktop with Kubernetes enabled
- VS Code terminal
- `kubectl` command-line tool

1. Create a Pod

kubectl run mypod --image=nginx:latest --port=80

Verify the Pod:

kubectl get pods

kubectl describe pod mypod

2. Create a Deployment

kubectl create deployment mydeployment --image=nginx:latest

Verify the Deployment:

kubectl get deployments

kubectl get pods

Scale the Deployment to three replicas:

kubectl scale deployment mydeployment --replicas=3

3. Create a Service

Expose the Deployment using a NodePort Service:

kubectl expose deployment mydeployment --type=NodePort --port=80

Verify the Service:

kubectl get services

kubectl describe service mydeployment

4. View all resources

kubectl get all

5. Delete the resources

kubectl delete service mydeployment

kubectl delete deployment mydeployment

kubectl delete pod mypod

Output

The Pod and Deployment are created successfully, and the Service exposes the Deployment's Pods. The resources can be verified using `kubectl get all`.

Result

Deployments, Services, and Pods were successfully created, inspected, and deleted using Kubernetes commands.

15. Create a Kubernetes service and Pod, and test them using Postman:

- **Pull the image from `https://hub.docker.com/r/ssatyab/studentmarkservice`.**
- **Download the following files:**
  - **`namespace.yml` — https://shorturl.at/4I3FI**
  - **`deployment.yml` — https://shorturl.at/HW960**
  - **`service.yml` — https://shorturl.at/G5NCK**

A. Theory

Kubernetes (K8s) is an open-source container orchestration platform used to deploy, manage, and scale containerized applications.

The main components used in this experiment are:

- Namespace: Provides a logical separation for Kubernetes resources.
- Pod: The smallest deployable unit in Kubernetes. It runs one or more containers.
- Deployment: Creates and manages Pods and maintains the desired number of replicas.
- Service: Exposes an application running inside Pods and provides a stable endpoint for accessing it.
- Docker Image: Contains the application and its required runtime environment.
- Postman: A tool used to send HTTP requests and test API responses.

In this experiment, the `ssatyab/studentmarkservice` Docker image is deployed using Kubernetes YAML configuration files. A Namespace, Deployment, and Service are created, and the application is tested using Postman.

1. Aim

To deploy the Student Mark Service application in a Kubernetes cluster using a Docker Hub image, configure a Namespace, Deployment, and Service, and test application endpoints using Postman.

2. Requirements

- Docker Desktop with Kubernetes enabled
- `kubectl` command-line tool
- Docker Hub image: `ssatyab/studentmarkservice`
- YAML configuration files: `namespace.yml`, `deployment.yml`, and `service.yml`
- Postman for sending HTTP requests

3. Theory

Kubernetes is a container orchestration platform used to deploy, manage, and scale containerized applications.

- Docker Image: Provides the packaged application and its runtime dependencies.
- Namespace: Creates a logical separation for Kubernetes resources.
- Pod: The smallest deployable unit in Kubernetes, containing one or more containers.
- Deployment: Manages Pods and maintains the desired number of replicas.
- Service: Provides a stable network endpoint to access application Pods.
- Postman: Allows testing APIs by sending HTTP requests and examining responses.

4. Project structure

Create a folder named `week15` containing these three files:

```
week15/
├── namespace.yml
├── deployment.yml
└── service.yml
```

Use the configuration files supplied by your instructor if available. The examples below reflect the configuration we have been testing; they are not verified copies of the instructor's original downloads.

5. Configuration files

A. `namespace.yml`

```
apiVersion: v1
kind: Namespace
metadata:
  name: studentmarks
```

This creates a separate namespace named `studentmarks`.

B. `deployment.yml`

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: studentmarkservice
  namespace: studentmarks
```

```
spec:
  replicas: 1
  selector:
    matchLabels:
      app: studentmarkservice
  template:
    metadata:
      labels:
        app: studentmarkservice
    spec:
      containers:
        - name: studentmarkservice
          image: ssatyab/studentmarkservice:latest
          imagePullPolicy: Always
          ports:
            - containerPort: 8100
```

This deployment pulls the Docker image and runs one replica of the application. The container port `8100` is based on the Spring Boot startup logs observed during testing.

C. `service.yml`

```
apiVersion: v1
kind: Service
metadata:
  name: studentmarkservice
  namespace: studentmarks
spec:
  type: NodePort
  selector:
    app: studentmarkservice
  ports:
    - protocol: TCP
      port: 8100
      targetPort: 8100
```

The Service selects Pods using the label `app: studentmarkservice` and forwards traffic to port `8100`. Kubernetes automatically assigns an available NodePort.

6. Procedure

Step 1: Open the project folder

Open the `week15` folder in VS Code. Create the three YAML files above and save them.

Step 2: Pull the Docker image

Open the VS Code terminal and run:

docker pull ssatyab/studentmarkservice:latest

Verify the image:

docker images

Step 3: Check Kubernetes

Ensure Docker Desktop is running and Kubernetes is enabled.

kubectl cluster-info
kubectl get nodes

The node should show a `Ready` status.

Step 4: Create the Namespace
kubectl apply -f namespace.yml

Expected output:

namespace/studentmarks created

If it already exists, Kubernetes may display `unchanged`.

Step 5: Deploy the application
kubectl apply -f deployment.yml

Step 6: Create the Service
kubectl apply -f service.yml

Step 7: Verify the resources
kubectl get all -n studentmarks

Check the Deployment:

kubectl get deployments -n studentmarks

Check the Pod:

kubectl get pods -n studentmarks

Check the Service:

kubectl get services -n studentmarks

The Pod should eventually show `Running` and `1/1` in the `READY` column.

If it does not start, inspect the logs:

kubectl logs deployment/studentmarkservice -n studentmarks

7. Test using Postman

Step 1: Forward the Service port

Run this command in the VS Code terminal:

kubectl port-forward service/studentmarkservice 8100:8100 -n studentmarks

Keep this terminal open while testing. Port forwarding provides local access to the Service.



Kubernetes
+1

Step 2: Test the application health

Open Postman and create a new HTTP request.

GET

`http://localhost:8100/actuator/health`

Select Send.

Expected response if the application is healthy:

```
{
  "status": "UP"
}
```

This tests application availability. It does not, by itself, test student-mark CRUD operations.

GET http://localhost:8100/actuator/mappings

1. Test the API using Postman

Base URL: `http://localhost:8100`

Test 1: Retrieve all students

GET

`http://localhost:8100/sms/findAll`

Click Send to retrieve all student records.

Test 2: Add a student

POST

`http://localhost:8100/sms/addOne`

1. Open the Body tab.
2. Select raw → JSON.
3. Enter this sample request body:

```
{

  "id": 101,

  "studentName": "Rahul",

  "totalMarks": 450

}
```

Click Send. The exact field names are based on your assignment's student data structure; the API's response will confirm whether the request is accepted.

Test 3: Retrieve a student by ID

GET

`http://localhost:8100/sms/findById?id=101`

Click Send. The `id=101` query parameter is a reasonable test based on the handler accepting an integer, but if it returns a parameter error, we will check the controller's exact parameter name.

Test 4: Update a student

PUT

`http://localhost:8100/sms/update`

Choose Body → raw → JSON and send:

```
{
  "id": 101,
  "studentName": "Rahul Sharma",
  "totalMarks": 475
}
```

Click Send.

Test 5: Delete a student

DELETE

`http://localhost:8100/sms/deleteById?id=101`

Click Send. If the parameter name is different, we will adjust it based on the API response.

2. Verify the result

After adding, updating, and deleting, run GET `/sms/findAll` again to verify the changes.

Operation	Method	Endpoint
View all students	GET	`/sms/findAll`
Add student	POST	`/sms/addOne`

Find by ID	GET	`/sms/findById`
Update student	PUT	`/sms/update`
Delete by ID	DELETE	`/sms/deleteById`

Step 3: Find the actual Student Mark API routes

The `/students` URL returned `404 Not Found` in your earlier test. The mappings response you shared showed Actuator routes, but did not establish the application's student-mark CRUD paths.

If the API documentation is available, use its listed paths and methods to test the actual student-mark operations. Do not invent POST, PUT, or DELETE URLs or request-body fields without confirming the API contract.

8. Expected output and observations

Component	Expected result
Docker image	`ssatyab/studentmarkservice` pulled
Namespace	`studentmarks` created
Deployment	One desired replica
Pod	`Running`, `1/1` ready

Service	`studentmarkservice` created
Port forwarding	Local port `8100` connected to Service port `8100`
Postman health test	HTTP `200 OK` with health status `UP`, if healthy
Student-mark API test	Depends on the actual API routes and request format

Take screenshots of the terminal showing the image pull and Kubernetes resources, and Postman showing the health response. Add successful CRUD screenshots only after the actual endpoints have been identified and tested.

9. Result

The Student Mark Service image is deployed in Kubernetes using a Namespace, Deployment, and NodePort Service. The application can be accessed locally through port forwarding and tested using Postman. The student-mark CRUD operations must be verified against the actual API routes provided by the image.

kubectl delete deployment <deployment-name> -n <namespace>
kubectl delete service <service-name> -n <namespace>